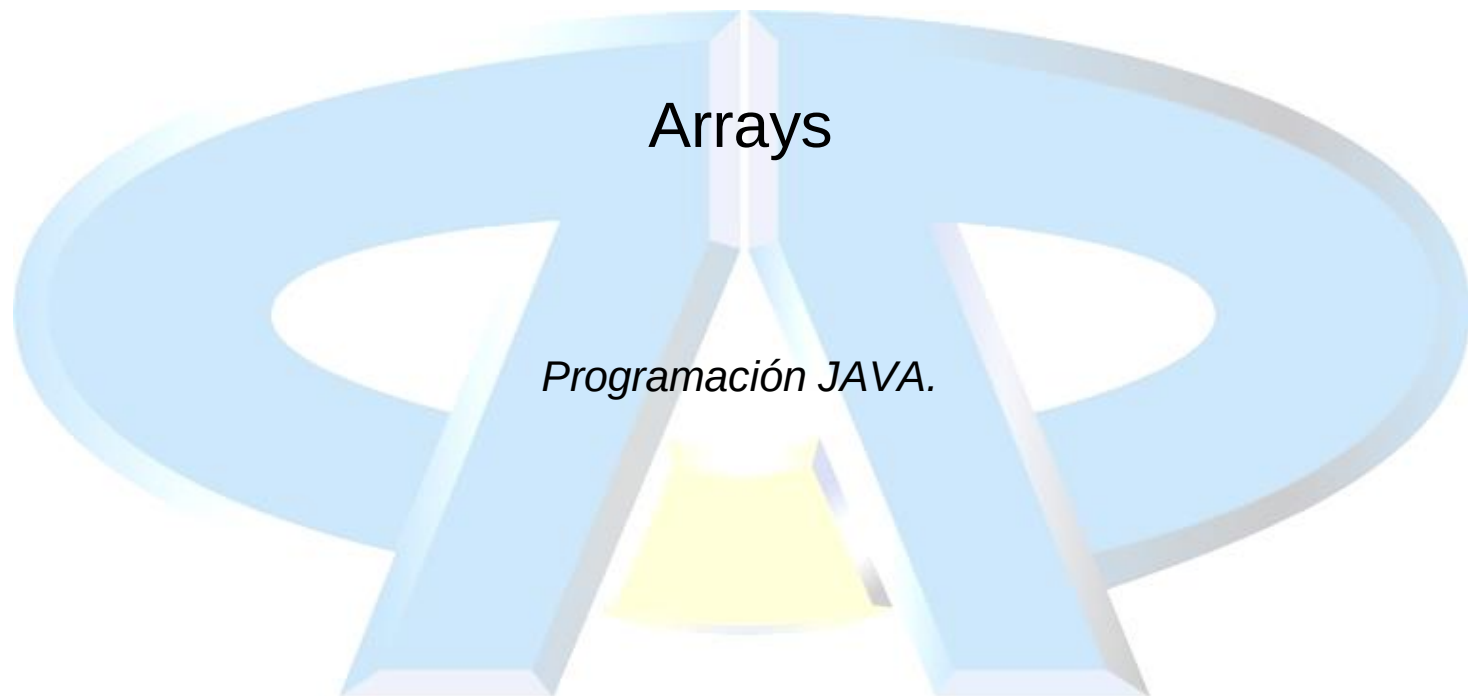
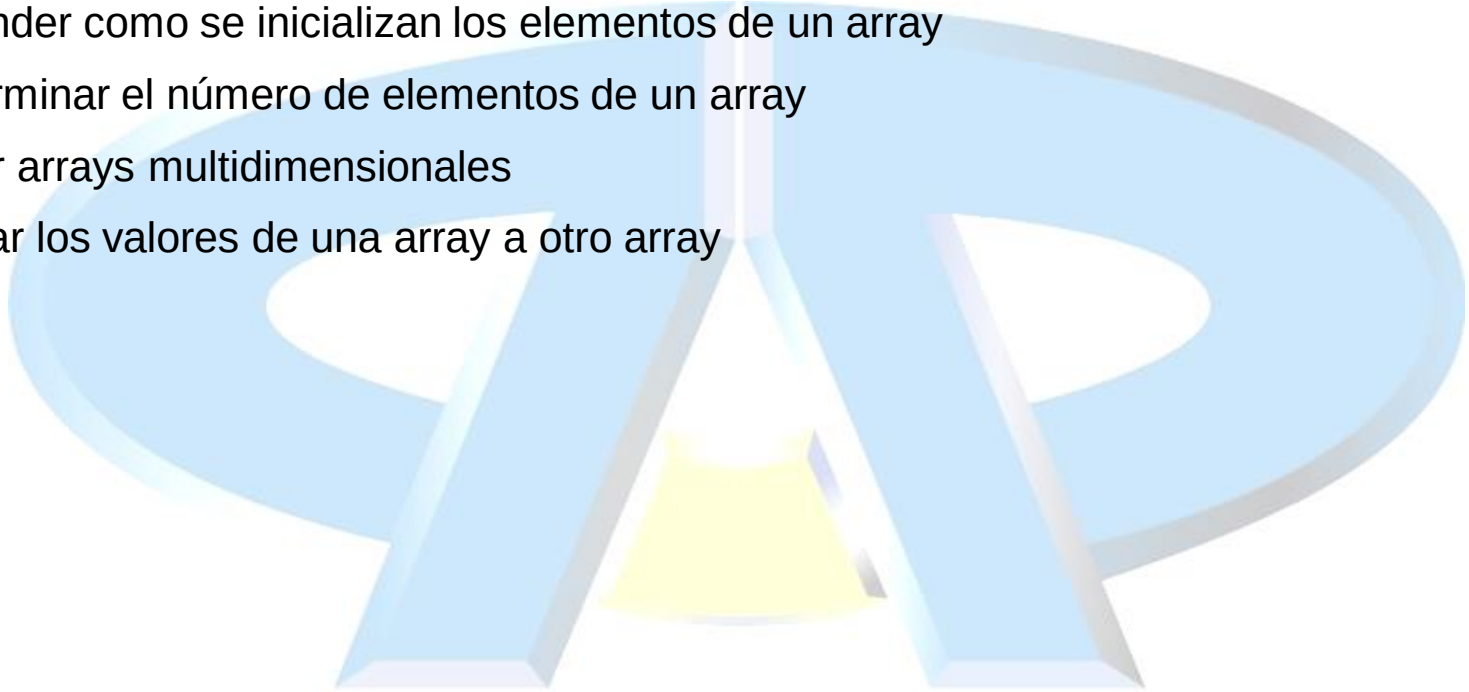




Objetivos

- Declarar y crear arrays de tipos de datos primitivos, clases o arrays
- Entender como se inicializan los elementos de un array
- Determinar el número de elementos de un array
- Crear arrays multidimensionales
- Copiar los valores de una array a otro array



Declaración de arrays

- Agrupación de elementos del mismo tipo, a los que podemos referirnos usando un nombre común.
- Un array es un objeto, por lo tanto debemos crearlo físicamente con *new* o con algún inicializador especial
- Se pueden declarar arrays de tipos primitivos o clases.

```
char[] s, t;           // Los corchetes afectan a las dos variables s y t
Point[] p1, p2;        // Los corchetes afectan a las dos variables p1 y p2
char s[], t;           // Los corchetes afectan solo a la variable s
Point p1[], p2;        // Los corchetes afecta solo a la variable p1
```

- Creación del array.
 - Declaración del array (puntero que apunta a un objeto de tipo “array”
`char[] s;`
 - Reserva de memoria inicialización del array (instanciación del objeto “array”)
`s = new char[25];`

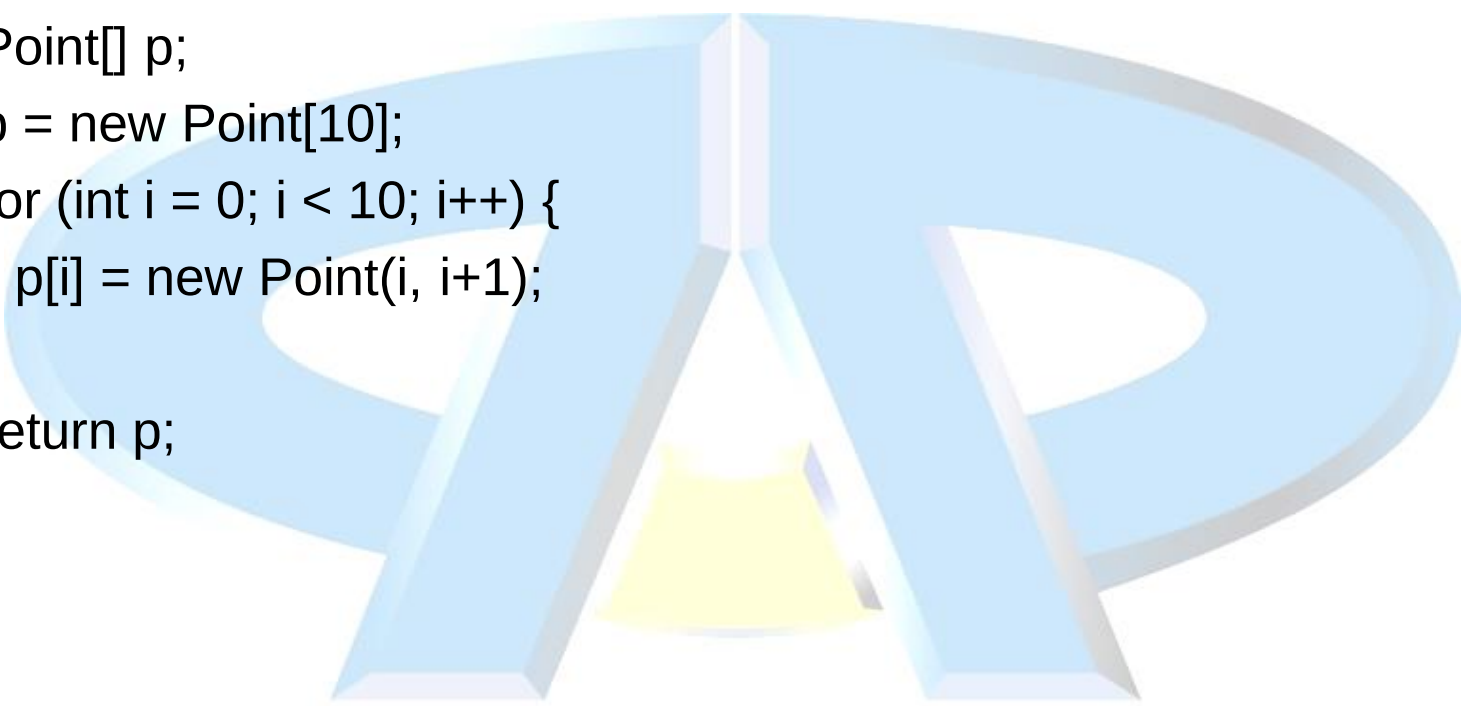
Ejemplo: Array de tipos primitivos

1. Se crea una variable de referencia (puntero) del tipo de array adecuado
2. Se instancia el array indicando su tamaño:
 - Se reserva memoria
 - Se inicializan al valor por defecto todas las posiciones del array
 - Se devuelve un apuntador, que en este caso se asigna a la variable de referencia declarada anteriormente

```
public char[] createArray() {  
    char []s;  
    s = new char[26];  
    for (int i=0; i < 26; i++) {  
        s[i] = (char) ('A' + i);  
    }  
    return s;  
}
```

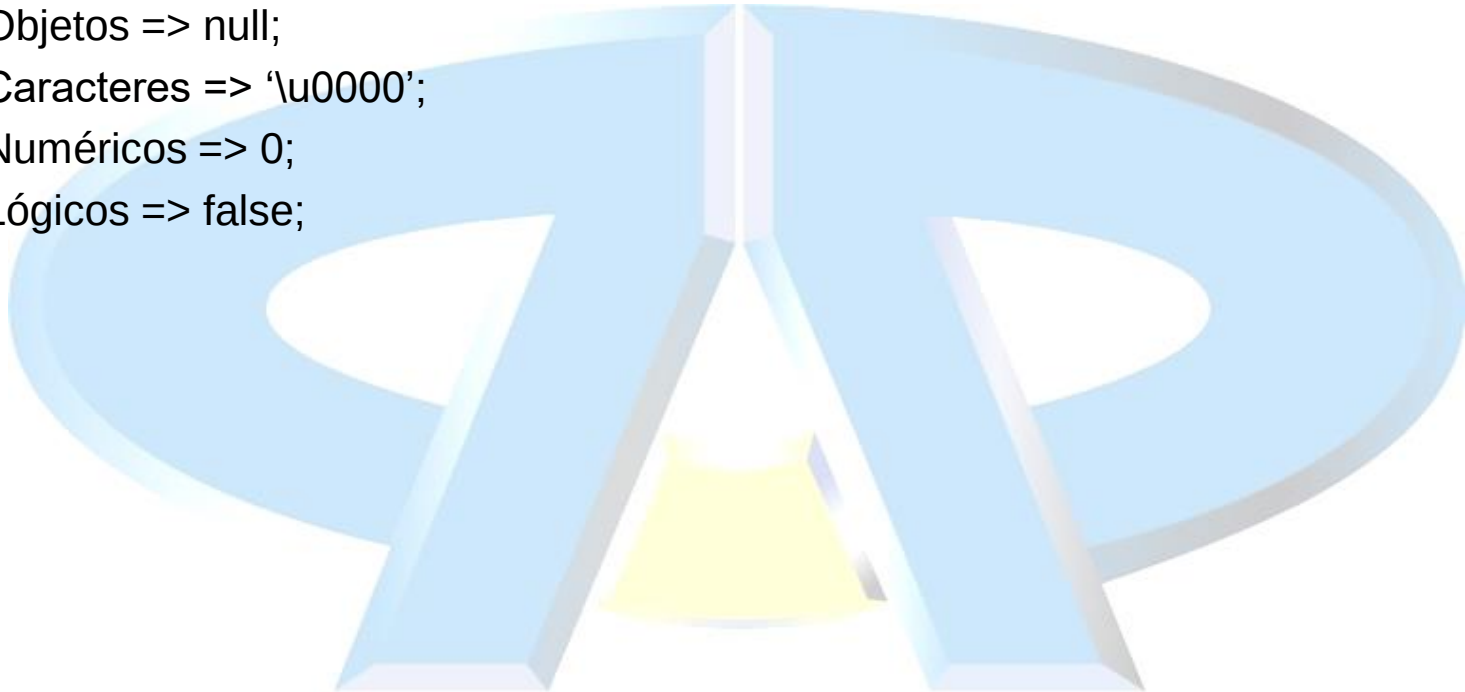
Ejemplo II: Array de Objetos

```
public Point[] createArray() {  
    Point[] p;  
    p = new Point[10];  
    for (int i = 0; i < 10; i++) {  
        p[i] = new Point(i, i+1);  
    }  
    return p;  
}
```



Inicialización de arrays.

- Un array se inicializa automáticamente durante su creación:
 - Objetos => null;
 - Caracteres => '\u0000';
 - Numéricos => 0;
 - Lógicos => false;

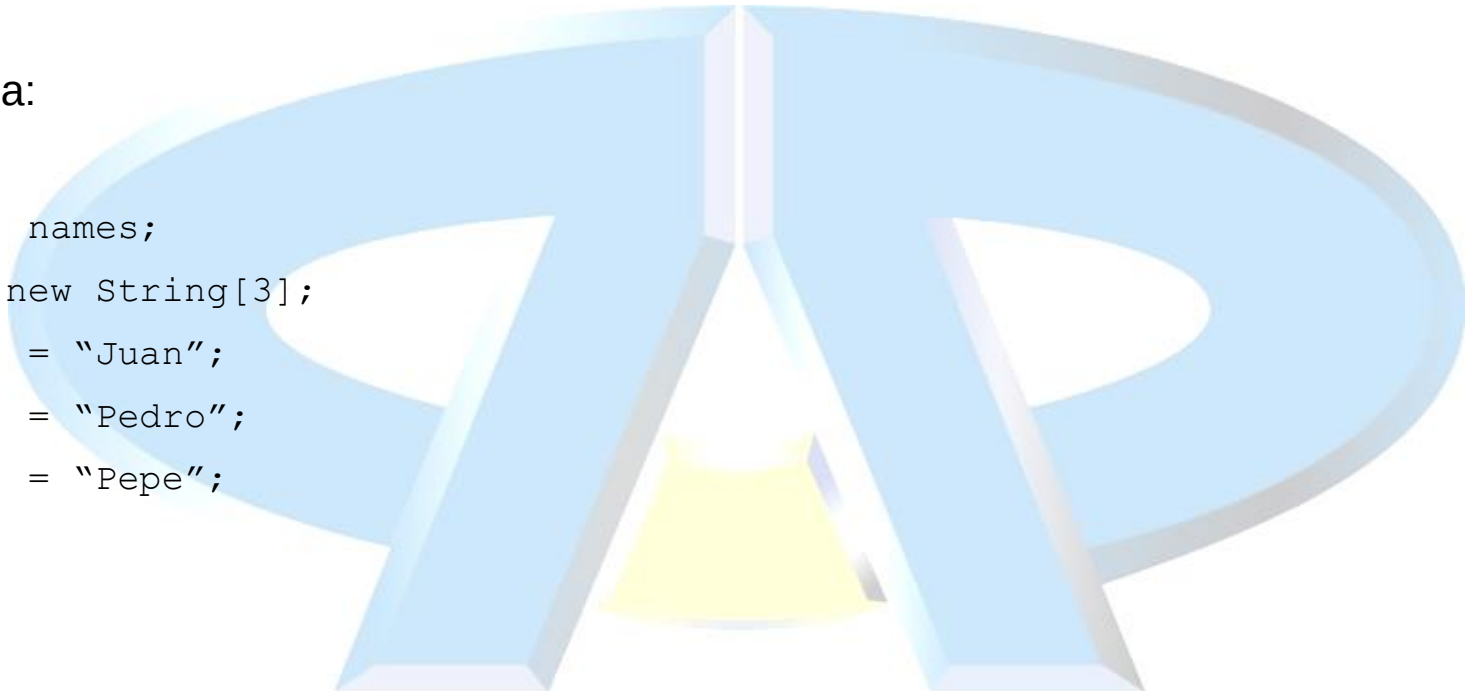


Inicialización Rápida

```
String[] names = {"Juan", "Pedro", "Pepe"}
```

Equivale a:

```
String[] names;  
names = new String[3];  
names[0] = "Juan";  
names[1] = "Pedro";  
names[2] = "Pepe";
```



Arrays n-dimensionales.

- En Java no existen los arrays n-dimensionales.
- Se pueden crear arrays de otro array => Sustituye a los arrays n-dimensionales.

```
int twoDim [][] = new int[4][];
```

```
twoDim[0] = new int [5];
```

```
twoDim[1] = new int [4];
```

```
int twoDim[][] = new int[][4]; //Error
```

```
int twoDim[][]= new int[6][4];
```


Límites de un array.

- Los índices de todos los arrays comienzan en 0
- Todos los arrays tienen un atributo público `length`
- Si nos salimos de rango se produce una excepción en tiempo de ejecución
- Se suele utilizar el atributo `length` para iterar sobre un array:

```
public void printElements(char[] list) {  
    for (int i=0; i < list.length; i++){  
        System.out.println(list[i]);  
    }  
}
```

- En la versión J2SE 5.0 se introdujo un bucle **for** que facilita la iteración sobre un bucle (se lee como “para cada elemento de la lista haz”)

```
public void printElements(char[] list) {  
    for (char element : list){  
        System.out.println(element);  
    }  
}
```

Redimensionamiento de arrays.

- Los arrays no se pueden redimensionar
- Si se puede hacer que una referencia a un array apunte a otro nuevo array
- ```
int[] myArray = new int[6];
```

 //Se declara la variable de referencia myArray  
//que apunta a un array de 6 posiciones
- ```
myArray = new int[10];
```

 //El puntero myArray apunta a un nuevo array,
//que en este caso tiene 10 posiciones.
//Evidentemente deja de apuntar al anterior

Copia de Arrays

- Java ofrece un método de la clase System que permite copiar arrays:

```
static void arraycopy (Object src, int srcPos, Object dest, int destPos,  
int length)
```

Copia un determinado número de elementos (length) desde un array origen (src), comenzando en la posición indicada (0), a un array destino (dest), a partir de la posición indicada (0)

```
// array original  
int[] myArray = {1, 2, 3, 4, 5, 6};  
// nuevo array, de tamaño mayor  
int[] hold = {10, 9, 8, 7, 6, 5, 4, 3, 2, 1};  
System.arraycopy(myArray, 0, hold, 0, myArray.length);
```

Clase utilidad Arrays

- En el paquete `java.util` existe la clase `Arrays` con métodos de utilidad estáticos:
 - `binarySearch`: Realiza una búsqueda de un elemento utilizando el algoritmo de búsqueda binaria, devolviendo la posición.
 - `copyOf`: Devuelve una copia de un array, con el número de elementos indicado. Si no hay suficientes elementos en el array original, se rellena con un valor adecuado al tipo del array.
 - `copyOfRange`: Devuelve una copia de un fragmento de un array
 - `equals`: Compara dos arrays devolviendo `true` cuando son iguales
 - `fill`: Rellena un array con un mismo elemento.
 - `hashCode`: Devuelve un `hashCode` para el array
 - `sort`: Realiza una ordenación ascendente de los elementos de un array
 - `toString`: Devuelve una representación en `String` del contenido de un array